

Universidade Federal de Santa Catarina (UFSC)

INE5425-06208 – Modelagem e Simulação

Professor: Dr. Rafael Luiz Cancian

Roteiro da Apresentação Beamer: Parser Dinâmico no GenESyS

Prévia para apresentação em simpósio

Vicente Cardoso dos Santos (22102203)
Ricardo Henrique Medeiros Pereira (22100637)
Eduardo Bischoff Grasel (22200355)

Sumário

1	Orientação Geral	1
2	Estrutura da Apresentação	1
3	Roteiro Slide a Slide	1
3.1	Slide 1 – Capa	1
3.2	Slide 2 – Roteiro da Prévia	2
3.3	Slide 3 – Problema: Modelo Extensível, Parser Fixo	2
3.4	Slide 4 – Objetivo	2
3.5	Slide 5 – Visão Geral da Solução	3
3.6	Slide 6 – Contratos Aproveitados e Completados	3
3.7	Slide 7 – Contrato de Mudanças	4
3.8	Slide 8 – Lazy Reload no Ciclo do Modelo	4
3.9	Slide 9 – Diagrama de Sequência: Lazy Reload	4
3.10	Slide 10 – Geração do Parser	5
3.11	Slide 11 – Diagrama de Componentes	5
3.12	Slide 12 – Compilação e Carregamento em Runtime	6
3.13	Slide 13 – Diagrama de Classes	7
3.14	Slide 14 – Substituição Segura do Parser	7
3.15	Slide 15 – Diagrama de Estados	7
3.16	Slide 16 – Plano de Testes	8
3.17	Slide 17 – Evidências de Teste	8
3.18	Slide 18 – Resultado Observado: Demonstração Demo	9
3.19	Slide 19 – Resultados do Projeto	9
3.20	Slide 20 – Encerramento	9
4	Sugestão de Distribuição de Tempo	10
5	Checklist de Coerência com os Slides	10

1 Orientação Geral

Este roteiro acompanha diretamente o arquivo `apresentacao_parser_dinamico.tex`. A numeração, os títulos e os blocos seguem a apresentação Beamer atual. Cada slide possui: descrição visual, conceitos que o aluno deve saber e uma sugestão de fala.

A apresentação está organizada como uma prévia de simpósio. Ela não tenta cobrir todo o relatório em detalhe; ela conduz o público por quatro perguntas: qual é o projeto, como ele foi desenvolvido, como foi testado e quais resultados foram obtidos.

2 Estrutura da Apresentação

Bloco	Slides	Função
Abertura	1–2	Apresentar o tema e o roteiro da prévia.
Projeto	3–5	Explicar o problema, o objetivo e a visão geral da solução.
Desenvolvimento	6–15	Explicar os contratos, o lazy reload, a geração, os diagramas e a substituição segura.
Testes	16–17	Mostrar o plano de testes e as evidências de validação.
Resultados	18–20	Mostrar a demonstração, os resultados e encerrar.

3 Roteiro Slide a Slide

Abertura

3.1 Slide 1 – Capa

Descrição do slide

Este slide corresponde à capa da apresentação Beamer. Ele deve mostrar o título “Mecanismo de Atualização Dinâmica do Parser no GenESyS”, o subtítulo “Prévia para apresentação em simpósio”, os nomes dos integrantes, a UFSC e a disciplina. A capa não deve exibir data.

Conceitos que o aluno deve saber

O aluno deve saber resumir o tema em uma frase: o trabalho permite que a linguagem de expressões do GenESyS seja atualizada em tempo de execução quando extensões do modelo declaram novas regras.

Script de fala sugerido

“Bom dia. Esta apresentação é uma prévia do nosso projeto sobre atualização dinâmica do parser no GenESyS. Vamos mostrar o problema, como a solução foi desenvolvida, como ela foi testada e quais resultados observamos.”

3.2 Slide 2 – Roteiro da Prévia

Descrição do slide

O slide mostra quatro blocos: apresentando o projeto, desenvolvimento, testes e resultados. Cada bloco aparece em um card curto, servindo como mapa da fala.

Conceitos que o aluno deve saber

Este slide organiza a narrativa. O público deve entender que a apresentação não começa diretamente no código; ela primeiro explica por que o problema existe.

Script de fala sugerido

“A apresentação está dividida em quatro partes. Primeiro apresentamos o projeto e o problema. Depois mostramos o desenvolvimento da solução. Em seguida, mostramos como testamos o mecanismo. Por fim, apresentamos os resultados e a demonstração.”

Bloco 1 – Apresentando o Projeto

3.3 Slide 3 – Problema: Modelo Extensível, Parser Fixo

Descrição do slide

O slide compara duas colunas: o modelo extensível e o parser original. Na parte inferior, há um fluxo indicando que uma nova extensão pode resultar em expressão não reconhecida se o parser continuar fixo.

Conceitos que o aluno deve saber

O parser original depende de regras já compiladas em Bison/Flex. Uma nova extensão pode existir no modelo sem ser reconhecida pela linguagem de expressões.

Script de fala sugerido

“Aqui está o desalinhamento. O modelo pode receber plugins e novas definições de dados, mas o parser original reconhece apenas aquilo que já foi compilado na gramática e no lexer. Então uma extensão pode existir no modelo, mas ainda assim a expressão correspondente pode falhar no parser.”

3.4 Slide 4 – Objetivo

Descrição do slide

O slide apresenta o objetivo central em um bloco: permitir que extensões declarem mudanças de linguagem e que o GenESyS atualize o parser automaticamente. Abaixo, quatro blocos resumem as etapas: declarar, agregar, gerar e conectar.

Conceitos que o aluno deve saber

Declarar mudanças é responsabilidade da extensão. Agregar, gerar e conectar o parser são responsabilidades do kernel. Essa separação evita que plugins tenham que lidar com Bison, Flex, compilação e carregamento dinâmico.

Script de fala sugerido

“O objetivo foi criar um fluxo em que a extensão declara o que muda na linguagem, e o GenESyS faz o restante. O kernel agrega as contribuições, gera os arquivos, compila uma biblioteca dinâmica e conecta o novo parser ao modelo.”

3.5 Slide 5 – Visão Geral da Solução

Descrição do slide

O slide exhibe o pipeline: extensão, `ParserChangesInformation`, marcação de *stale*, `parseExpression`, `ParserManager`, Bison/Flex+C++, biblioteca `.so` e novo parser ativo.

Conceitos que o aluno deve saber

Stale significa que o parser atual ainda funciona, mas pode estar desatualizado. O reload acontece sob demanda, na próxima avaliação de expressão.

Script de fala sugerido

“Este é o fluxo geral. Uma extensão entrega um pacote de mudanças. O modelo marca o parser como possivelmente desatualizado. Quando uma expressão for avaliada, o `ParserManager` gera e compila uma nova biblioteca e conecta o novo parser ao modelo.”

Bloco 2 – Desenvolvimento

3.6 Slide 6 – Contratos Aproveitados e Completados

Descrição do slide

O slide tem duas colunas: “Já previsto” e “Implementado”. A primeira lista contratos e marcadores existentes; a segunda lista o que foi concretizado, como campos léxicos, `hasChanges()`, agregação, geração da `.so` e carregamento dinâmico.

Conceitos que o aluno deve saber

Contrato é uma convenção arquitetural que permite que partes do sistema conversem sem conhecer detalhes concretos. O trabalho completou uma arquitetura já prevista, em vez de criar um parser paralelo.

Script de fala sugerido

“Um ponto importante é que o projeto já tinha sinais de uma arquitetura para parser dinâmico. Existiam contratos e marcadores. O desenvolvimento consistiu em completar esses pontos e fazer o fluxo realmente sair da extensão e chegar ao parser carregado.”

3.7 Slide 7 – Contrato de Mudanças

Descrição do slide

O slide apresenta `ParserChangesInformation` como pacote de fragmentos. A figura divide os fragmentos em Bison, Flex e `hasChanges()`.

Conceitos que o aluno deve saber

`ParserChangesInformation` não compila nem executa nada; ele transporta fragmentos. Tokens e produções pertencem ao Bison. Regras léxicas pertencem ao Flex. `hasChanges()` evita recompilar quando não há mudança real.

Script de fala sugerido

“A extensão comunica suas necessidades por meio de `ParserChangesInformation`. Esse objeto carrega fragmentos de gramática e de lexer. Ele também informa se há mudança real, para que objetos vazios não disparem recompilações desnecessárias.”

3.8 Slide 8 – Lazy Reload no Ciclo do Modelo

Descrição do slide

O slide mostra dois momentos: a inserção da extensão apenas marca o parser como stale; o primeiro uso por `parseExpression` dispara o reload automático.

Conceitos que o aluno deve saber

Lazy reload adia o custo de recompilação até o momento em que a linguagem precisa ser usada. Isso evita regenerar o parser várias vezes durante a montagem do modelo.

Script de fala sugerido

“A escolha foi fazer reload sob demanda. Quando uma extensão entra, o sistema não recompila imediatamente. Ele apenas marca o parser como stale. A recompilação acontece quando uma expressão é avaliada, que é o momento em que a linguagem precisa estar atualizada.”

3.9 Slide 9 – Diagrama de Sequência: Lazy Reload

Descrição do slide

O slide mostra o fluxo temporal entre extensão, `ModelDataManager`, `Model`, `ParserManager`, `toolchain`, biblioteca `.so` e parser.

Conceitos que o aluno deve saber

Diagrama de sequência mostra a ordem das chamadas. A extensão fornece mudanças; o modelo dispara a atualização; o `ParserManager` gera e conecta o parser; depois a expressão é finalmente avaliada.

Script de fala sugerido

“Este diagrama mostra a mesma ideia em ordem temporal. Primeiro a extensão entra no modelo e fornece mudanças. Depois, quando uma expressão é avaliada, o modelo pede ao ParserManager para agregar, gerar e conectar o novo parser. Só depois disso a expressão é avaliada.”

3.10 Slide 10 – Geração do Parser

Descrição do slide

O slide mostra que os arquivos-base `bisonparser.yy` e `lexerparser.ll` continuam sendo a fonte de verdade. Os fragmentos são injetados por marcadores e geram arquivos modificados.

Conceitos que o aluno deve saber

Bison gera o parser a partir da gramática. Flex gera o scanner a partir de regras léxicas. A solução não cria uma nova linguagem do zero; ela injeta fragmentos nos pontos planejados.

Script de fala sugerido

“A geração preserva os arquivos-base como fonte de verdade. O ParserManager apenas injeta os fragmentos da extensão nos marcadores corretos. Assim, a linguagem original continua central, e o parser dinâmico nasce como uma versão modificada dela.”

3.11 Slide 11 – Diagrama de Componentes

Descrição do slide

O slide mostra o mecanismo dividido em três regiões visuais. A região da esquerda representa as extensões do modelo. A região central representa o kernel do GenESyS. A região da direita representa a geração do parser e o carregamento em runtime.

Na região de extensões, a caixa `ModelDataDefinition` representa qualquer elemento ou plugin que pode participar do modelo. O ponto importante é que esse elemento não compila parser, não chama Bison e não carrega biblioteca. Ele apenas fornece informações de mudança por meio do hook de extensão. A caixa `ParserChangesInformation` representa exatamente esse pacote de informações: tokens, produções gramaticais e regras léxicas.

Na região do kernel, a caixa `Model` representa o modelo de simulação que recebe extensões e avalia expressões. A caixa `ParserManager` é o coordenador do mecanismo: ele agrega mudanças, gera os arquivos modificados, invoca a compilação e conecta o novo parser. A caixa `Parser_if` representa a interface pela qual o restante do simulador usa o parser ativo. Essa interface é importante porque o modelo não precisa saber se o parser ativo é o parser original ou um parser gerado dinamicamente. A caixa `ParserDefaultImpl2` representa a implementação concreta que efetivamente avalia expressões.

Na região de geração e runtime, as caixas `bisonparser.yy` e `lexerparser.ll` representam os arquivos-base da linguagem. A caixa de arquivos modificados representa as versões geradas depois da injeção dos fragmentos. A toolchain Bison/Flex/compilador C++ transforma esses arquivos em uma biblioteca compartilhada `genesys_parser_dynamic.so`. Depois, `dlopen` abre essa biblioteca, `dlsym` encontra a factory `genesys_createParser`, e essa factory cria uma nova instância de `ParserDefaultImpl2`.

Conceitos que o aluno deve saber

Diagrama de componentes mostra responsabilidades e fronteiras. A fronteira entre extensão e kernel separa quem declara mudanças de quem aplica mudanças. A fronteira entre kernel e toolchain separa a lógica do simulador das ferramentas externas de geração e compilação. A fronteira entre biblioteca dinâmica e runtime mostra que o novo parser não nasce como objeto comum do executável original; ele é carregado a partir de uma `.so`.

O aluno também deve saber explicar as setas principais. A seta de `ParserChangesInformation` para `ParserManager` significa que o kernel recebe fragmentos declarados pela extensão. As setas para os arquivos modificados significam injeção nos arquivos Bison/Flex. A seta da toolchain para `genesys_parser_dynamic.so` significa compilação de uma biblioteca compartilhada. A seta de `dlopen/dlsym` para a factory significa carregamento e busca da função de criação. A seta do novo parser para `Parser_if` significa que o parser gerado continua obedecendo à interface esperada pelo modelo.

Script de fala sugerido

Este diagrama é importante porque ele mostra a arquitetura inteira separada por responsabilidades. Começando pela esquerda, temos a extensão do modelo. Ela pode ser qualquer elemento derivado de `ModelDataDefinition` que queira alterar a linguagem. Essa extensão não compila nada; ela apenas devolve um `ParserChangesInformation` com tokens, produções e regras léxicas.

No centro está o kernel do GenESyS. O `Model` é quem usa expressões, mas quem coordena o mecanismo é o `ParserManager`. Ele recebe as mudanças, combina as contribuições e manda gerar o novo parser. O `Parser_if` aparece aqui porque o modelo usa o parser por interface. Isso permite substituir o parser ativo sem mudar o restante do simulador.

Na direita está o pipeline de geração. O `ParserManager` parte dos arquivos-base do Bison e do Flex, injeta os fragmentos da extensão e produz arquivos modificados. Depois, Bison, Flex e o compilador C++ geram a biblioteca `genesys_parser_dynamic.so`. Por fim, em runtime, o sistema usa `dlopen` para abrir essa biblioteca e `dlsym` para encontrar a função `genesys_createParser`. Essa factory cria o novo `ParserDefaultImpl2`, que volta para o modelo pela interface `Parser_if`. Assim, a extensão apenas declara, o kernel coordena, a toolchain gera e o runtime carrega o novo parser.

3.12 Slide 12 – Compilação e Carregamento em Runtime

Descrição do slide

O slide traz dois blocos: toolchain e runtime. Toolchain: Bison, Flex e compilador C++. Runtime: `dlopen()`, `dlsym()` e `genesys_createParser()`.

Conceitos que o aluno deve saber

Uma biblioteca `.so` pode ser carregada em runtime. `dlopen` abre a biblioteca. `dlsym` busca a factory. `extern "C"` fornece um nome estável, sem *name mangling*.

Script de fala sugerido

“Depois da injeção, Bison e Flex geram código C++, e o compilador cria uma biblioteca compartilhada. Em runtime, usamos `dlopen` para abrir a biblioteca e `dlsym` para localizar

a factory C, que cria o novo parser.”

3.13 Slide 13 – Diagrama de Classes

Descrição do slide

O slide mostra as classes principais: `Model`, `ParserManager`, `ModelDataManager`, `ModelDataDefinition`, `ParserChangesInformation`, `Parser_if`, `ParserDefaultImpl2` e `ParserFactory`.

Conceitos que o aluno deve saber

`Model` possui o parser ativo e o gerenciador. `ModelDataDefinition` fornece o hook. `ParserManager` agrega, gera e conecta. `Parser_if` isola o restante do simulador da implementação concreta.

Script de fala sugerido

“No diagrama de classes, vemos como os contratos se conectam. O `Model` possui o `ParserManager` e o parser ativo. As definições de dados podem fornecer mudanças. O `ParserManager` usa essas mudanças para gerar a biblioteca e a factory cria a implementação concreta pela interface `Parser_if`.”

3.14 Slide 14 – Substituição Segura do Parser

Descrição do slide

O slide mostra parser antigo, `Sampler_if` e parser novo. As setas indicam `releaseSampler()` e `setSamplerOwned()`.

Conceitos que o aluno deve saber

Ownership define quem deve destruir um objeto. O `Sampler_if` precisa ser transferido para o novo parser sem dupla liberação e sem vazamento.

Script de fala sugerido

“Trocar o parser não é apenas trocar um ponteiro. O parser usa um sampler, e esse objeto precisa sobreviver à troca. Por isso, o parser antigo libera o sampler sem destruí-lo, e o novo parser assume a posse.”

3.15 Slide 15 – Diagrama de Estados

Descrição do slide

O slide mostra os estados *Clean*, *Stale*, *Generating*, *Connected* e *Failed*. A transição de erro volta para *Clean* preservando o parser antigo.

Conceitos que o aluno deve saber

O estado *Failed* representa falha controlada. A falha pode acontecer em Bison, Flex, compilação, `dlopen` ou `dlsym`; em todos esses casos, o parser antigo deve permanecer usável.

Script de fala sugerido

“O diagrama de estados mostra a robustez do mecanismo. O modelo começa com um parser válido. Quando há mudanças, ele fica stale. Se a geração funciona, o novo parser é conectado. Se falha, o erro é retornado e o parser antigo continua preservado.”

Bloco 3 – Testes

3.16 Slide 16 – Plano de Testes

Descrição do slide

O slide divide os testes em três blocos: contrato, integração e falhas. Cada bloco tem três itens curtos.

Conceitos que o aluno deve saber

Testes de contrato validam objetos isolados. Testes de integração validam o fluxo completo. Testes de falha validam robustez e preservação do parser anterior.

Script de fala sugerido

“A validação foi organizada em três grupos. Primeiro, verificamos contratos como `ParserChangesInformation`. Depois, testamos o fluxo completo do parser dinâmico. Por fim, testamos falhas para garantir que o parser antigo não seja perdido.”

3.17 Slide 17 – Evidências de Teste

Descrição do slide

O slide atual não usa mais uma tabela apertada. Ele mostra seis blocos em duas colunas: fluxo principal, atualização sob demanda, múltiplas extensões, falha controlada, biblioteca ausente e ownership.

Conceitos que o aluno deve saber

Cada bloco corresponde a um conjunto de testes. Os nomes completos dos testes podem ser citados se houver pergunta, mas no slide aparecem rótulos legíveis para facilitar a apresentação.

Script de fala sugerido

“Aqui estão as principais evidências. O fluxo principal comprova que uma extensão pode adicionar uma função. O reload sob demanda comprova que a atualização acontece no primeiro uso. Também testamos múltiplas extensões, falhas controladas, biblioteca ausente e transferência segura do sampler.”

Bloco 4 – Resultados

3.18 Slide 18 – Resultado Observado: Demonstração Demo

Descrição do slide

O slide mostra três imagens: antes da extensão, inserção da extensão e depois da atualização. A leitura principal é que `demo()` não era reconhecida antes e passou a ser reconhecida depois.

Conceitos que o aluno deve saber

A demonstração Demo é uma evidência visual. Ela mostra, de forma simples, uma extensão declarando token, regra Flex e produção Bison para que a linguagem passe a reconhecer uma função nova.

Script de fala sugerido

“Como resultado visual, usamos a demonstração Demo. Antes, a função `demo` não era reconhecida. Depois, a extensão foi inserida no modelo. Na avaliação seguinte, o parser foi atualizado e a função passou a ser aceita.”

3.19 Slide 19 – Resultados do Projeto

Descrição do slide

O slide lista os resultados: contratos funcionais, declaração de mudanças léxicas e sintáticas, parser regenerado sob demanda, biblioteca carregada por factory C e falhas preservando o parser anterior.

Conceitos que o aluno deve saber

O resultado principal é arquitetural: a linguagem deixa de ser fixa e passa a acompanhar extensões carregadas no modelo. O mecanismo não substitui Bison/Flex; ele automatiza seu uso.

Script de fala sugerido

“Os principais resultados são estes: os contratos previstos passaram a formar um fluxo funcional; plugins podem declarar mudanças; o parser é regenerado sob demanda; a biblioteca é carregada em runtime; e falhas preservam o parser anterior.”

3.20 Slide 20 – Encerramento

Descrição do slide

O slide final mostra “Obrigado”, o tema e a chamada para perguntas.

Conceitos que o aluno deve saber

Este slide não introduz conceito novo. Ele deve reforçar a mensagem final: o parser agora acompanha dinamicamente as extensões do modelo.

Script de fala sugerido

“Com isso, encerramos a prévia. A ideia central é que o parser do GenESyS agora pode acompanhar dinamicamente as extensões carregadas no modelo. Obrigado. Estamos abertos a perguntas.”

4 Sugestão de Distribuição de Tempo

Slides	Foco	Tempo sugerido
1–2	Abertura e roteiro	1–2 min
3–5	Apresentando o projeto	3–4 min
6–15	Desenvolvimento e diagramas	8–10 min
16–17	Testes	3 min
18–20	Resultados e fechamento	3 min

5 Checklist de Coerência com os Slides

- O roteiro possui 20 slides, a mesma quantidade da apresentação Beamer.
- Os títulos do roteiro acompanham os títulos dos frames em `apresentacao_parser_dinamico.tex`.
- O slide de testes foi atualizado para combinar com os blocos visuais da apresentação, não com a tabela antiga.
- Os diagramas de sequência, componentes, classes e estados aparecem no roteiro nos mesmos pontos em que aparecem nos slides.
- A capa do roteiro e a capa da apresentação não exibem data.